

*While you
Caffeinate*



JS

async / await

While you **caffeinate** serie
Offered by ☕ Arnaud **Buchholz**



An **async** function **always** returns a  **promise**.

```
async function hello() {  
  return 'world';  
}
```

// is equivalent to

```
function hello() {  
  return Promise.resolve('world');  
}
```

Any **uncaught exception** occurring inside the function body is automatically converted into a **rejected promise**.

```
async function hello() {  
  return transform('world');  
}
```

// is equivalent to

```
function hello() {  
  try {  
    return Promise.resolve(transform('world'));  
  } catch (error) {  
    return Promise.reject(error);  
  }  
}
```

Important notes

An **async** function does not **magically** become **asynchronous** :
it solely depends on the **function body**.

Every rejected promise **must** be handled.

```
> Promise.reject('KO')  
< ▶ Promise {<rejected>: 'KO'}  
✖ ▼ Uncaught (in promise) KO  
   (anonymous) @ VM123:1
```

The **await** keyword is available only* inside an **async** function.

It waits for a **promise** to be **settled** :

- When **fulfilled**, the value is **returned** and the next statement **executed**.
- When **rejected**, the **await** keyword **throws** an exception with the **rejection reason**.

* Inside a module, you might use a  **top level await**

await can be used with **any** value but it makes sense mostly with promises.

```
async function examples () {  
  console.log(await 'Hello World!'); // valid  
}
```

It works with  **Thenable** objects.

Sequential vs Parallel

```
async function sequential () {  
  const result1 = await asyncMethod1();  
  const result2 = await asyncMethod2(); // asyncMethod1 completed  
}
```

```
async function parallel () {  
  const promise1 = asyncMethod1();  
  const promise2 = asyncMethod2(); // asyncMethod1 still in progress  
  const result1 = await promise1;  
  const result2 = await promise2;  
}
```

```
async function parallelAndDestructuring () {  
  const [result1, result2] = await Promise.all([  
    asyncMethod1(),  
    asyncMethod2()  
  ]);  
}
```